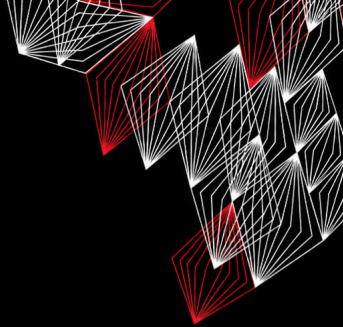


Tema 103: Comandos GNU y Unix

Erick Ricardo Martinez Martinez
Luis Fernando Torres Hernández

Laboratorio de Supercómputo y Visualización
en Paralelo

Junio 21, 2026



Contenido

- 1 103.1 Trabajar desde la línea de comandos
- 2 103.2 Procesar secuencias de texto usando filtros
- 3 103.3 Administración básica de archivos
- 4 103.4 Uso de secuencias de texto, tuberías y redireccionamientos
- 5 103.5 Crear, supervisar y matar procesos
- 6 103.6 Modificar la prioridad de ejecución de los procesos
- 7 103.7 Búsquedas en archivos de texto usando expresiones regulares
- 8 103.8 Edición básica de archivos
- 9 Preguntas y comentarios

Obteniendo información del sistema

Comandos básicos

`pwd`: Imprime la ruta del directorio de trabajo actual.

`uname -a`: Muestra toda la información del sistema (kernel, hostname, arquitectura, SO).

Ejemplo

```
$ pwd
/home/frank
$ uname -a
Linux base 4.18.0-18-generic #19~18.04.1-Ubuntu SMP
x86_64 x86_64 x86_64 GNU/Linux
```

Opciones de `uname`

`-s`: Nombre del kernel

`-n`: Hostname de red

`-r`: Release del kernel

`-m`: Arquitectura de hardware

`-o`: Sistema operativo

Obteniendo información de los comandos

Documentación y localización de comandos

`man COMANDO`: Abre la página de manual (requiere el nombre exacto).

`apropos PALABRA`: Busca comandos por palabra clave cuando no se recuerda el nombre exacto.

`type CMD`: Indica si un comando es binario, alias, función o *shell builtin*, y si está *hashed*.

`which CMD`: Muestra solo la ruta absoluta del binario (útil en scripts).

`help`: Muestra información sobre comandos *builtin* de `bash`.

Ejemplo: `apropos`

```
$ apropos kernel  
systemd-udevd-kernel.socket (8) - Device event managing daemon  
uname (2) - get name and information about current kernel
```

Nota

`Bash` mantiene una tabla *hash* de comandos usados recientemente para localizarlos más rápido. Se limpia con `hash -d`.

Ejemplo: type y which

Identificando comandos

```
$ type uname cp kill which
uname is hashed (/bin/uname)
cp is /bin/cp
kill is a shell builtin
which is /usr/bin/which
```

```
$ which uname which
/bin/uname
/usr/bin/which
```

Diferencia clave

type clasifica el comando (binario, builtin, alias, hashed).
which solo devuelve la ruta absoluta del binario.

Usando el historial de comandos

El historial de Bash

`history`: Lista los comandos ejecutados en la sesión actual (base de datos dinámica).

`history N`: Muestra solo las últimas N líneas.

`~/.bash_history`: Archivo donde se guarda el historial; los comandos más recientes solo se escriben ahí al cerrar la sesión.

Flechas **arriba/abajo**: Navegan y recuperan comandos previos para reejecutarlos.

Ejemplo

```
$ ls -a /home/frank  
. .. .bash_history .bash_logout .bashrc .profile .ssh
```

Variables de entorno

Conceptos

Una **variable de entorno** es un valor preestablecido del shell, consultable con \$NOMBRE.

env: Muestra solo las variables **exportadas**.

set: Muestra **todas** las variables y funciones, incluidas las locales.

Una variable local (var=valor) solo existe en el shell actual; con export se propaga a los shells hijos.

Variable local vs. exportada

```
$ myvar=hello
$ bash
$ echo $myvar

$ exit
$ export myvar
$ bash
$ echo $myvar
hello
```

Crear y eliminar variables de entorno

Comandos

`VAR=valor`: Crea una variable local (sin espacios alrededor de =).

`export VAR`: Exporta la variable a los subshells.

`unset VAR`: Elimina una variable.

Ejemplo

```
$ mynewvar=goodbye
$ env | grep mynewvar
$ set | grep mynewvar
mynewvar=goodbye
```

```
$ unset mynewvar
$ echo $mynewvar
```

Importante

No debe haber espacios a ambos lados del signo igual al asignar variables: `var=valor`, nunca `var = valor`.

Manteniendo el valor de caracteres especiales

Comillas y escape

Comillas **dobles** ("..."): Conservan el valor literal de todo excepto \$, ' y \.

Comillas **simples** ('...'): Conservan el valor literal absoluto de todos los caracteres.

Escape (\): Antepuesto a un carácter especial, hace que se lea literalmente.

Ejemplo: nombres de archivo con espacios

```
$ touch my big file
```

```
$ ls
```

```
my big file          (crea TRES archivos)
```

```
$ touch "my big file"
```

```
$ ls
```

```
'my big file'        (un solo archivo)
```

```
$ echo "$mynewvar"  
goodbye
```

```
$ echo '$mynewvar'  
$mynewvar
```

Introducción: filosofía Unix y filtros

Filosofía Unix (Doug McIlroy)

Programas que hacen una sola cosa bien, trabajan con flujos de texto y se combinan entre sí.

Canales estándar

stdin: Entrada estándar (teclado por defecto).

stdout: Salida estándar (pantalla por defecto).

stderr: Salida de error estándar (pantalla por defecto).

Repaso rápido

> sobrescribe, >> agrega, < usa un archivo como entrada, | conecta la salida de un comando con la entrada de otro.

Visualizando el contenido de archivos

Comandos de lectura

`cat` archivo: Concatena/muestra el contenido completo.
`less` archivo: Pagina el archivo con navegación tipo vi.
`head -n N` archivo: Primeras N líneas (10 por defecto).
`tail -n N` archivo: Últimas N líneas (10 por defecto).
`tail -f` archivo: Sigue el archivo en tiempo real (útil para logs).

Ejemplo

```
$ sudo tail -n 5 /var/log/syslog
$ tail -f /var/log/syslog | grep -i 'manufacturer'
Nov 8 06:01:35 host kernel: usb 1-4.3: Manufacturer: SanDisk
```

Leyendo archivos comprimidos

Lectura sin descomprimir

`zcat archivo.gz`: Lee archivos comprimidos con `gzip`.
`bzcat archivo.bz2`: Lee archivos comprimidos con `bzip2`.
`xzcat archivo.xz`: Lee archivos comprimidos con `xz`.
`gzip archivo`: Comprime el archivo (crea `.gz` y borra el original).
`gunzip archivo.gz`: Descomprime un archivo `gzip`.

Ejemplo

```
$ gzip ftu.txt
$ ls ftu*
ftu.txt.gz
$ zcat ftu.txt.gz | grep -v cat
cut
head
less
```

sed: el editor de flujo (uso básico)

Sintaxis e instrucciones

`sed -n /patron/p < archivo`: Imprime solo las líneas coincidentes (`-n` suprime impresión automática).

`sed /patron/d < archivo`: Elimina (no imprime) las líneas coincidentes.

`sed s/viejo/nuevo/ < archivo`: Sustituye la primera ocurrencia por línea.

`sed -i.backup s/x/y/ archivo`: Edita el archivo *in-place*, guardando respaldo con sufijo `.backup`.

Ejemplo

```
$ sed -n /cat/p < ftu.txt
```

```
bzcat
```

```
cat
```

```
xzcat
```

```
zcat
```

```
$ sed s/cat/dog/ < ftu.txt
```

```
bzdog
```

```
dog
```

Volcado de datos y sumas de comprobación

od (octal dump)

od archivo: Volcado en octal (por defecto).
-x: Volcado en hexadecimal. -c: Muestra caracteres.

Sumas de comprobación (checksums)

md5sum, sha256sum, sha512sum: Calculan/verifican la integridad de un archivo mediante una función hash.

Ejemplo

```
$ sha256sum ftu.txt > sha256.txt  
$ sha256sum -c sha256.txt  
ftu.txt: OK
```

```
$ echo "new entry" >> ftu.txt  
$ sha256sum -c sha256.txt  
ftu.txt: FAILED
```

Comandos

`cut -d:` `-f1`: Extrae campos; `-d` delimitador, `-f` número(s) de campo.

`paste -d, f1 f2`: Une archivos en columnas; `-d` cambia el delimitador (tab por defecto).

`tr 'a' 'b'`: Traduce caracteres; `-s` comprime repeticiones, `-d` elimina.

Ejemplos

```
$ grep ":/bin/bash$" /etc/passwd | cut -d: -f1
```

```
$ paste firstname lastname department | tr '\t' ,
```

```
$ cut -d: -f3 /etc/passwd | sort -u | wc -l
```

split, sort, uniq y un ejemplo combinado

Comandos

`split -l 100 -d archivo prefijo`: Divide un archivo en partes de 100 líneas con sufijos numéricos.

`sort -u`: Ordena líneas eliminando duplicados. `sort -R`: Orden aleatorio.

`uniq -c`: Filtra líneas **adyacentes** repetidas y cuenta ocurrencias.

Ejemplo combinado: solo nombres de carpetas en /etc

```
$ ls -l /etc | grep ^d | tr -s ' ' | cut -d" " -f9,3
```

Nota

`uniq` solo detecta líneas adyacentes repetidas; si no lo son, hay que ordenar primero con `sort`.

Conceptos clave

Un **archivo** almacena datos/programas; tiene contenido y metadatos (tamaño, propietario, fecha, permisos).

Un **directorio** es un archivo especial que almacena otros archivos.

Primer carácter de `ls -l`: - archivo normal, d directorio, c archivo especial.

Ruta absoluta: comienza con `/`. **Ruta relativa:** respecto al directorio actual.

Comandos básicos: ls y touch

ls

-l: Formato de lista larga. -h: Tamaños legibles (con -l).
-a: Incluye archivos ocultos. -R: Listado recursivo.

touch

Crea archivo(s) vacío(s) o cambia timestamps. -a: solo hora de acceso. -m: solo hora de modificación.

Ejemplo

```
$ ls -lh
total 60K
drwxr-xr-x 2 frank frank 4.0K Apr  1 2018 Desktop
-rw-r--r-- 1 frank frank  21 Sep  7 12:59 emp_name
```

Copiar, mover y eliminar archivos

Comandos

`cp ORIGEN DESTINO`: Copia archivos.

`mv ORIGEN DESTINO`: Mueve o renombra archivos. `-i` pide confirmación, `-f` sobrescribe sin preguntar.

`rm ARCHIVO`: Elimina archivos. `-i` pide confirmación, `-f` fuerza sin preguntar.

Ejemplo

```
$ mv -i old_file_name new_file_name
```

```
mv: overwrite 'new_file_name'?
```

```
$ rm -i file1
```

```
rm: remove regular file 'file1'?
```

Advertencia

Tenga mucho cuidado con `rm -rf`: un borrado recursivo en un directorio importante del sistema puede inutilizarlo. Úselo solo cuando esté absolutamente seguro del contenido a eliminar.

Crear y eliminar directorios

Comandos

`mkdir DIRECTORIO`: Crea directorio(s). `-p`: Crea estructura de padres/hijos.
`rmdir DIRECTORIO`: Elimina un directorio solo si está **vacío**. `-p`: Elimina estructura de padres/hijos vacíos.

Ejemplo

```
$ mkdir -p parents/children  
$ rmdir -p parents/children
```

Diferencia clave

`rmdir` solo borra directorios vacíos; `rm -r` borra independientemente de si tienen contenido.

Manipulación recursiva de archivos y directorios

Recursividad

Aplicar la misma acción a un directorio y a todo su árbol de subdirectorios (-r/-R/--recursive).

Ejemplo

```
$ tree mydir
```

```
mydir
|_file1
|_newdir
   |_file2
```

```
$ cp mydir newcopy
```

```
cp: omitting directory 'mydir'
```

```
$ cp -r mydir newcopy          (ahora si copia todo el arbol)
```

```
$ rm -r newcopy/
```

Archivos Globbing y Wildcards

Globbing

Caracteres especiales del shell que representan múltiples nombres de archivo.

Tipos de Wildcards

Wildcard	Significado
*	Cero, una o más ocurrencias de cualquier carácter
?	Una sola ocurrencia de cualquier carácter
[]	Cualquier carácter encerrado (acepta rangos, ej. [0-9], [A-Z])

Ejemplos de Wildcards

Ejemplos

```
$ ls  
last.txt lest.txt list.txt third.txt past.txt
```

```
$ ls l[ae]st.txt  
last.txt lest.txt
```

```
$ ls l[a-z]st.txt  
last.txt lest.txt list.txt
```

```
$ ls [plf]?st*  
last.txt lest.txt list.txt past.txt
```

```
$ ls f*[0-9].txt  
file1.txt file23.txt fom23.txt
```

Encontrando archivos: find

Sintaxis general

`find RUTA_INICIAL OPCIONES EXPRESION`

Opciones principales

<code>-name / -iname</code>	Busca por nombre (sensible / insensible a mayúsculas)
<code>-type f/d/l</code>	Archivos regulares / directorios / enlaces simbólicos
<code>-mtime N</code>	Modificado hace N días
<code>-size +100k</code>	Mayor a 100 KB (c=bytes, k=KB, M=MB, G=GB)
<code>-empty</code>	Archivos vacíos
<code>-exec CMD {} \;</code>	Ejecuta CMD sobre cada resultado
<code>-delete</code>	Elimina los archivos encontrados

Ejemplos con find

Ejemplos

```
$ find /home/frank -name "*.png"  
/home/frank/Pictures/logo.png  
$ sudo find /var -size +2G  
/var/lib/libvirt/images/debian10.qcow2  
$ find . -name "*.conf" -exec chmod 644 '{}' \;  
$ find . -name "*.bak" -delete
```

Importante

Encierre {} en comillas simples. Use -delete solo cuando esté seguro de los resultados.

Archivado de archivos: tar

Sintaxis

```
tar [OPERACION_Y OPCIONES] [ARCHIVO] [ARCHIVOS]
```

Operaciones y opciones

-c	Crea un nuevo archivo tar (--create)
-x	Extrae archivo(s) (--extract)
-t	Lista el contenido (--list)
-v	Modo verboso (--verbose)
-f	Especifica el nombre del archivo (--file=)
-z / -j	Comprime/descomprime con gzip / bzip2
-C DIR	Extrae el contenido en el directorio DIR

Ejemplos: tar, cpio y dd

tar

```
$ tar -czvf archivo.tar.gz stuff
$ tar -xvf archivo.tar -C /tmp
```

cpio

```
$ ls | cpio -o > archivo.cpio
$ cpio -id < archivo.cpio
```

dd (copia a bajo nivel)

```
$ dd status=progress if=/dev/sda of=backup.dd bs=4096
```

Descriptores de archivo

Canales estándar

Canal	FD	Por defecto
stdin	0	Teclado
stdout	1	Pantalla
stderr	2	Pantalla

Redirección

Reasignación del descriptor de archivo de un canal mediante caracteres especiales en la línea de comandos.

Operadores de redirección

Operador	Significado
> o 1>	Redirige stdout a archivo (sobrescribe)
>>	Redirige stdout a archivo (agrega)
<	Redirige contenido de archivo a stdin
2>	Redirige stderr a archivo (sobrescribe)
2>>	Redirige stderr a archivo (agrega)
&>	Redirige stdout y stderr al mismo destino
2>&1	Redirige stderr a stdout

Cuidado

No deje espacio junto al & en &>, o el shell lo interpretará como ejecución en segundo plano.

noclobber y /dev/null

noclobber

Opción de Bash que evita sobrescribir archivos existentes mediante redirección de salida.

`set -o noclobber` (o `set -C`): activa.

`set +o noclobber` (o `set +C`): desactiva.

Ejemplo

```
$ set -o noclobber
```

```
$ cat /proc/cpu_info 2>/tmp/error.txt
```

```
-bash: /tmp/error.txt: cannot overwrite existing file
```

/dev/null

Archivo especial que descarta cualquier dato escrito en él; no afectado por noclobber.

tee: mostrar y guardar a la vez

`tee [-a] archivo`

Muestra la salida en pantalla y además la guarda en un archivo. `-a` agrega en lugar de sobrescribir.

Ejemplo: capturando stderr con tee

```
$ make | tee log.txt
make: *** No targets specified... Stop.
$ cat log.txt
                                     (log.txt queda vacio)

$ make 2>&1 | tee log.txt
$ cat log.txt
make: *** No targets specified... Stop.
```

Nota

Solo el stdout de un proceso es capturado por una tubería; para capturar también stderr hay que redirigirlo antes con `2>&1`.

Tuberías (pipes) y sustitución de comandos

Tuberías (|)

Conectan la salida de un comando directamente a la entrada de otro. El shell inicia todos los comandos simultáneamente; los datos fluyen de izquierda a derecha.

Sustitución de comandos

Reemplaza un comando por su salida estándar usando 'comando' o \$(comando).

Ejemplo

```
$ mkdir $(date +%Y-%m-%d)
```

```
$ OS=$(uname -o)
```

```
$ echo $OS
```

```
GNU/Linux
```


Here Document y Here String

Conceptos

Here Document (<<TERM ... TERM): redirección de entrada multilínea hasta encontrar un término final.

Here String (<<<"cadena"): redirección de entrada de una sola línea.

Ejemplo

```
$ wc -c <<EOF
> How many characters
> in this Here document?
> EOF
43
```

```
$ wc -c <<< "How many characters in this Here string?"
41
```

xargs: construyendo comandos desde stdin

xargs

Construye y ejecuta comandos usando elementos recibidos por stdin como argumentos.

- n N: usa N argumentos por ejecución. -L N: usa N líneas por ejecución.
- 0: usa el carácter nulo como separador (combinar con `find -print0`).
- I TERM: define un término de sustitución para ubicar el argumento en cualquier posición.

Ejemplo: mover archivos evitando espacios en nombres

```
$ find . -mindepth 2 -name '*avi' -print0 \  
  | xargs -0 -I PATH mv PATH ./
```

Control de trabajos (jobs)

Conceptos

Job: proceso iniciado interactivamente desde una terminal, enviado a segundo plano, aún no finalizado.

Jobspec: especificación de trabajo (%1, %nombre, %+, %-).

Comandos

comando &: ejecuta en segundo plano. Ctrl+Z: suspende. Ctrl+C: termina.

jobs [-l]: lista trabajos activos (-l muestra el PID).

fg [%n] / bg [%n]: trae a primer plano / envía a segundo plano.

Señales y terminación de procesos

Identificar procesos

`pgrep nombre / pidof nombre`: obtiene PID(s) por nombre.

Terminar procesos

`kill PID`: envía SIGTERM (siempre requiere PID o jobspec).

`kill -9 PID (-KILL)`: fuerza terminación cuando otras señales fallan.

`pkill nombre / killall nombre`: termina procesos por nombre.

`nohup comando &`: desliga el proceso de la sesión (sobrevive al cierre de sesión).

Ejemplo

```
$ kill -1 $(pgrep sleep)
```

Comandos de estado general

`uptime`: tiempo de actividad, usuarios conectados, *load average* (1, 5, 15 min).

`free`: uso de memoria (total, used, free, shared, buff/cache, swap).

`watch -n SEG comando`: ejecuta un comando periódicamente (2s por defecto).

ps vs. top

`ps`: instantánea estática de procesos. `top`: monitoreo dinámico en tiempo real.

ps: estilos de opciones

Estilo	Sintaxis	Ejemplo
BSD	sin guion	<code>ps a</code> , <code>ps U carol</code>
UNIX	un guion	<code>ps -u carol</code>
GNU	doble guion	<code>ps --user carol</code>

ps aux (estilo BSD)

```
USER  PID  %CPU  %MEM  VSZ   RSS TTY  STAT  START  TIME  COMMAND
root    1   0.0   0.1  9292   ...  ?    Ss    08:00  0:02  systemd
```

Personalizar columnas

```
ps -o user,pid,%mem,%cpu,cmd --sort=-pcpu
```

top: interacción y columnas

Teclas interactivas

M: ordenar por memoria
P: ordenar por %CPU
k: matar proceso
r: renice
u: filtrar por usuario

Columnas clave

PR: prioridad de kernel
NI: valor nice
S: estado (S, R, Z)
%CPU / %MEM

Alternativas

htop (más amigable) y atop (más exhaustivo).

Multiplexores de terminal

Concepto

Permiten cambiar entre múltiples sesiones/ventanas dentro de un solo terminal, manteniéndolas activas tras desconectarse (*detach*).

GNU Screen vs. tmux

Característica	Screen	tmux
Prefijo predeterminado Ctrl+a	x	
Modelo cliente-servidor		x
Matar región no mata su ventana	x	
Sesiones se pueden separar (detach)	x	x

GNU Screen: comandos clave

Prefijo: Ctrl+a

Ctrl+a c: nueva ventana Ctrl+a n/p: ventana siguiente/anterior
Ctrl+a S / |: dividir región horizontal/vertical
Ctrl+a d: desvincular sesión (*detach*)

Gestión de sesiones

```
$ screen -S "trabajo"  
$ screen -ls  
$ screen -r trabajo
```

Configuración

/etc/screenrc (sistema) o ~/.screenrc (usuario).

tmux: comandos clave

Prefijo: Ctrl+b

Ctrl+b c: nueva ventana Ctrl+b % / ": dividir panel vertical/horizontal
Ctrl+b flechas: moverse entre paneles Ctrl+b z: zoom de panel
Ctrl+b d: desvincular sesión

Gestión de sesiones

```
$ tmux new -s "trabajo"  
$ tmux ls  
$ tmux attach -t trabajo
```

Diferencia clave

Matar un panel en tmux SÍ mata su pseudo-terminal y programas asociados (a diferencia de las regiones en Screen).

Conceptos

Un sistema **multitarea** ejecuta más de un proceso "simultáneamente" (real con varias CPU, o simulado por cambio rápido de contexto).

Un sistema **preventivo (preemptive)**: el SO puede quitar la CPU a un proceso en ejecución para dársela a uno de mayor prioridad.

El **planificador (scheduler)** organiza la cola de procesos según su política y prioridad.

Políticas de programación

Tiempo real: prioridad estática decide todo, siempre supera a procesos normales.

Normal: usa el valor *nice* para la prioridad dinámica.

Prioridad estática y Niceness

Prioridad estática

Procesos de **tiempo real**: rango 0-99.

Procesos **normales**: rango 100-139 (39 niveles).

Menor número = mayor prioridad.

Niceness (valor nice)

Predicado adicional de los procesos normales: rango **-20** (alta prioridad, "menos agradable") a **19** (baja prioridad, "más agradable"). Valor por defecto: **0** (prioridad 120).

nice y renice

Comandos

`nice -n VALOR comando`: ejecuta un comando con un nuevo valor nice (10 por defecto si se omite `-n`).

`renice VALOR -p PID`: cambia el valor nice de un proceso ya en ejecución.

`renice +5 -g grupo / -u usuario`: aplica a todos los procesos de un grupo o usuario.

Ejemplo

```
$ nice -n 15 tar czf home_backup.tar.gz /home
```

```
# renice -10 -p 2164
```

```
2164 (process ID) old priority 0, new priority -10
```

Importante

Solo **root** puede reducir el valor nice de un proceso por debajo de cero.

Consultando prioridades con ps y top

`ps -el: columnas PRI y NI`

```
F S UID PID PPID C PRI NI ADDR SZ WCHAN TTY TIME CMD
4 S  0  1    0 0  80  0  -  9292  -   ? 00:00:00 systemd
```

Fórmula de ps

Prioridad real = valor PRI mostrado + 40 (ej. $80 + 40 = 120$ = prioridad normal estándar)

Error común

```
renice -10 21704 sin privilegios: Permission denied.
```

Introducción a las expresiones regulares

Conceptos

Expresión regular (RE): patrón genérico para localizar/modificar texto.

Átomo: elemento básico de una RE (un carácter con o sin significado especial).

Ancla: coincide con una posición (^ inicio, \$ fin de línea), no con un carácter.

RE Básicas (BRE) vs. RE Extendidas (ERE)

Estándar POSIX: difieren en qué metacaracteres necesitan escape con \. `grep` usa BRE por defecto; `egrep/grep -E` usa ERE.

Expresión de corchetes y clases POSIX

Expresión de corchetes

[abc]	Coincide con a, b o c
[^abc]	Coincide con cualquier carácter EXCEPTO a, b, c
[0-9], [a-z]	Rangos de dígitos o letras

Clases de caracteres POSIX (dentro de [])

[:digit:]	Dígito (0-9)
[:alpha:]	Carácter alfabético
[:alnum:]	Alfanumérico
[:space:]	Espacio en blanco
[:upper:] /	Mayúscula / minúscula
[:lower:]	

Cuantificadores y límites: BRE vs. ERE

Cuant.	Significado	ERE	BRE
*	0 o más repeticiones	*	*
+	1 o más repeticiones	+	\+
?	0 o 1 repetición	?	\?
{i,j}	entre i y j veces	{i,j}	\{i,j\}
	alternancia (A o B)		\
()	agrupación / backreference	()	\(\)

Regla de coincidencia más larga

Si una subcadena coincide y otra más larga que empieza en el mismo punto también coincide, se toma la más larga.

grep, egrep y fgrep

Variantes

egrep = grep -E (RE extendidas) fgrep = grep -F (búsqueda literal)

Opciones importantes

-c	Solo el número de líneas coincidentes
-i	No distingue mayúsculas/minúsculas
-n	Muestra el número de línea
-v	Selecciona líneas que NO coinciden
-o	Muestra solo la parte coincidente
-C N	N líneas de contexto antes/después

Ejemplos de grep

Ejemplos

```
# fdisk -l | grep '^Disk /dev/sd[ab]' -c
```

```
2
```

```
$ egrep "[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}"
```

```
$ grep -v ^# /etc/services
```

```
$ egrep ".org$|.com$" domains.txt
```

Sintaxis

[DIRECCION] COMANDO [OPCIONES] Dirección: número de línea, rango (1,7) o RE entre /.../.

Instrucciones clave

d	Elimina la(s) línea(s) coincidentes
c TEXTO	Reemplaza la línea coincidente con TEXTO
a TEXTO	Agrega TEXTO después de la coincidencia
s/A/B/	Sustituye la primera coincidencia de A por B
s/A/B/g	Sustituye TODAS las coincidencias en la línea

Ejemplos de sed

Ejemplos

```
$ factor `seq 12` | sed "1d;/:.*2.*/d"
```

```
3: 3
```

```
5: 5
```

```
7: 7
```

```
9: 3 3
```

```
11: 11
```

```
$ sed -e "/^#/d" /etc/services
```

Nota

Sin `-r/-E`, sed usa BRE: los paréntesis de agrupación deben escaparse como `\(` y `\)`.

Combinando grep y sed

Caso real: intentos fallidos de inicio de sesión (/var/log/btmp)

lastb: lee intentos fallidos de login. -d: resuelve IP a hostname.

Ejemplo

```
# lastb -d -a --time-format notime \  
| grep -v '[0-9]$\' \  
| sed -e 's/.* \(.*\)$/\1/' \  
| sort | uniq
```

Conceptos

vi: editor de texto estándar, preinstalado, basado en atajos de teclado.

vim (vi mejorado): compatible con vi; añade resaltado de sintaxis, undo/redo multinivel, edición de múltiples documentos.

Modo Normal: modo predeterminado al abrir vi; teclas asociadas a comandos.

Modo Inserción: el texto escrito aparece directamente en pantalla; se sale con Esc.

Invocación

`vi archivo` `vi +N archivo (línea N)` `vi + archivo (última línea)`

Modo Normal: navegación

O / \$	inicio / fin de línea	h j k l	izq / abajo / arriba / der
1G / G	inicio / fin del documento	w / W	saltar palabra
(/)	inicio / fin de oración	/ / ?	buscar adelante / atrás
{ / }	inicio / fin de párrafo	e / E	fin de palabra actual

Repetición numérica

Un número antes del comando lo repite N veces, ej. d5w elimina la palabra actual + 4 siguientes.

Modo Normal: edición, copiado y pegado

i / a	insertar antes/después del cursor	x	eliminar carácter
I / A	insertar inicio/fin de línea	u	deshacer
o / O	nueva línea abajo/arriba + insertar	Ctrl+R	rehacer
yy / p	copiar / pegar línea	dd	eliminar línea
r	reemplazar carácter	ZZ	guardar y cerrar

Ejemplo

3yy copia la línea actual + 2 siguientes. dt. elimina desde el cursor hasta el siguiente punto.

Comandos de dos puntos (:)

<code>:s/RE/TEXT0/g</code>	Reemplaza todas las coincidencias en la línea actual
<code>:! </code>	Ejecuta un comando de shell
<code>:q / :q!</code>	Salir / salir sin guardar
<code>:wq</code>	Guardar y salir
<code>:x</code>	Guardar y salir si es necesario

Recursos de ayuda

`:help vim-modes-intro` `vimtutor`: tutorial interactivo paso a paso.

GNU nano

Editor simple: todas las pulsaciones se insertan directamente (no hay modos).

`Ctrl+K`: cortar selección `Ctrl+U`: pegar

`Meta+U` / `Meta+E`: deshacer / rehacer

Emacs

Editor de texto + IDE completo (compila, ejecuta, prueba programas). El texto se inserta directamente, pero la navegación usa comandos de teclado.

Configurando el editor predeterminado

Variables de entorno

VISUAL o EDITOR definen el editor predeterminado del shell (usado, por ejemplo, por `crontab -e`).

Ejemplo

```
$ export EDITOR=nano
```

Persistencia

Agregar la línea a `~/.bash_profile`.

¿Preguntas o comentarios?